

NSF ERC



Dexterous Design Challenge

Project Module 08

Arduino & C++

Module Overview
Agenda

01

Introduction to C++

Programming Fundamentals

02

Introduction to Arduino

Connecting Code to Hardware

03

Libraries in Arduino

Using Existing Code

01

Introduction to C++

Comments

Comments document your code and help explain what it does. The compiler ignores comments when compiling your program.

Single-Line

Use // for a short comment.

```
// Single line comment
```

This is also used to title sections of code to organize it for other users.

Multi-Line

Use /* */ to enclose a larger comment.

```
/*   Multi
     Line
     Comment  */
```

This is also used to temporarily “comment out” code to stop it from running when testing other sections.

Storing Data in Variables

In C++, variables store data. Each variable must have a declared data type and be declared before use.

Type	Use	Example
int	Integer	-1, 4, 27
float	Non-Integer Number	3.14, -5.8
char	Character	‘A’, ‘b’, ‘5’
string	Multiple Characters	“hello”, “world”
bool	Boolean; TRUE or FALSE	TRUE, FALSE

Variable Declaration

Variables are declared by the sequence:

type name = value;

```
int example = 1;
```

‘example’ has the value of 1

Variable Naming Conventions

Case Sensitive

`example` and `Example` are different.

Allowed Characters

Only letters, numbers or underscores can be used in variable names.

Valid Beginning

Variables can only begin with and underscore or letter. No numbers.

No Reserved Words

Reserved keywords such as TRUE and FALSE cannot be used as variable names.

Example Variable Names

`sensor-value` `temperature1` `int` `motor_speed` `joint angle`

Math and Assignment Operators

C++ operators perform operations on variables and values, including basic mathematical calculations.

Operator	Meaning	Example (x = 10, y = 3)	Result
+	Add	x + y	13
-	Subtract	x - y	7
*	Multiply	x * y	30
/	Divide	x / y	3 ①
%	Modulus	x % y	1

① Division returns an integer value, ignoring any decimal or remainder.

Math and Assignment Operators

C++ also has various other math operations that can help reduce the amount of code written.

Operator	Meaning	Example (x = 10, y = 3)	Result
++	Increment	++ x	11
--	Decrement	-- y	2
+=	Assignment	x += y	x = x + y ①

① Assignment operators exist for all arithmetic operations.

Comparison Operators

C++ comparison operators evaluate relationships between values and return TRUE (1) or FALSE (0).

Operator	Meaning	Example (x=5, y=3)	Result
== ①	Equal to	x == y	FALSE (0)
!=	Not equal to	x != y	TRUE (1)
>	Greater than	x > y	TRUE (1)
<	Less than	x < y	FALSE (0)
>=	Greater than or equal to	x >= y	TRUE (1)
<=	Less than or equal to	x <= y	FALSE (0)

① Single and double = have distinct meanings. Mixing them will result in inoperable code.

Logical Operators

Logical operators evaluate values and return TRUE (1) or FALSE (0). Only 0 is considered FALSE; any nonzero value is TRUE.

Operator	Meaning	Example (x = 1, y = 0, z = 3)	Result
&&	AND	x && y	FALSE (0)
II	OR	z || y	TRUE (1)
!	NOT	!x	FALSE (0)

Strings

A string is a sequence of characters enclosed in double quotes.

Valid String	Invalid String
“Hello World!”	Hello World!

Printing a String

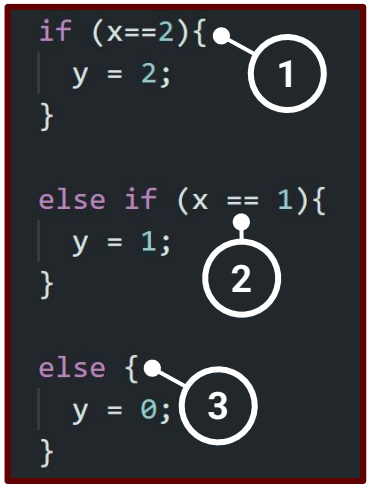
Strings can be printed on the current line using: `Serial.print(variable);`

Strings can be printed on their own line using: `Serial.println(variable);`

Printing strings can be used to display live updates in code during use.

Conditionals - If Statement

Coding often has situations where certain sections of code should only be run under specific circumstances. (E.g. a computer unlocking only when a correct password is entered). The most commonly used conditional is an **if/else** conditional:



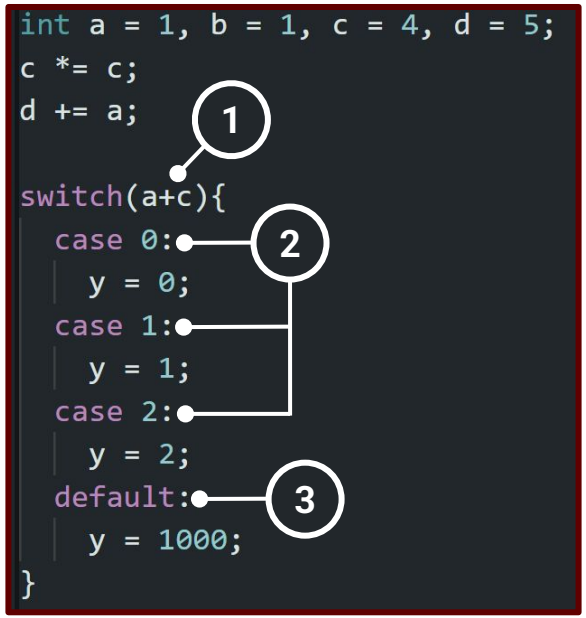
1. The expression being evaluated is placed here. In this case, we are looking at the value of 'x==2'. If true, the code in this block is run.

2. If the previous expression was false, the next expression to be evaluated is here. If true, the code in this block is run

3. The 'else' code runs whenever no 'if' or 'else if' block runs.

Conditionals - Switch Statement

The second type of conditional in C++ is the **switch**. Switches are used when there are a large number of cases:



1. The expression being evaluated is placed here. In this case, we are looking at the value of ‘a+c’.

2. After ‘a+c’ is evaluated, it is compared to each case value. If there is a match, it executes that code segment.

3. If there isn’t, the program moves onto the default case. Default cases are not mandatory and can be removed if all values of the switch expression are known and given a case.

Loops

Loops

Loops

Functions

Functions

Functions

Functions

Functions

Scope

02

Introduction to Arduino

What is Arduino?

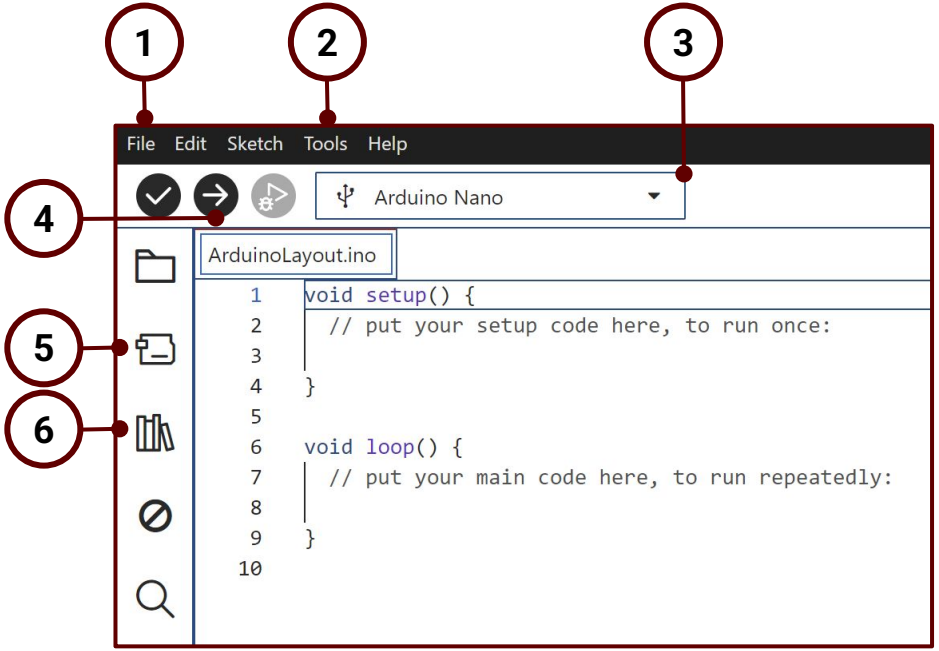
Arduino is an open source company that makes the following resources:

Arduino Microcontrollers - Microcontrollers are hardware devices with a small computer and the peripherals needed to run programs. Arduinos are a type of microcontroller board called a development board. Development boards are designed for projects and education, and have additional peripherals such as GPIOs, built in LEDs, buttons, etc.

Arduino IDE - The Arduino IDE, (integrated development environment), is the software that communicates code to a microcontroller.

Arduino Programming Language - The Arduino programming language is unique to Arduino based microcontrollers. It is largely based on C++, with a few structural changes to make it better suited for microcontroller use.

IDE Layout



1. File Menu
2. Tools Menu
3. Board Selection
4. Upload
5. Board Manager
6. Library Manager

Setting Up Arduino

1 Plug in the Arduino

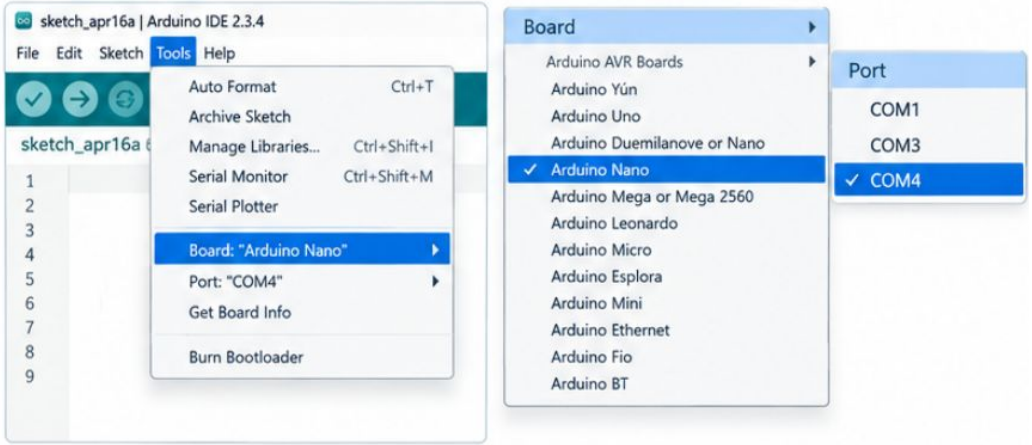
Use the provided USB-C cable to connect your Arduino Nano to your laptop.

2 Check the COM Port

Open Device Manager on your laptop and check which COM Port is assigned to your Arduino

3 Select the Board and COM Port in the Arduino IDE

Open the Arduino IDE and set the correct board and COM Port from the Tools menu.



Syntax

All Arduino programs, known as sketches, have two functions: setup and loop. Additional functions, variable definitions, or library declarations can be included, but a sketch will not complete without these two functions.

```
// Global variable declarations go here
// Function definitions go here

void setup() {
  // put your setup code here, to run once:
}

void loop() {
  // put your main code here, to run repeatedly:
}
```

When an Arduino sketch starts, the setup function is run a single time.

All code that is not part of setup should go in the loop function. This will run indefinitely until the program is stopped.

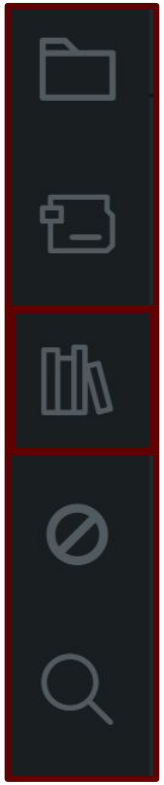
Function definitions, and global variable declarations should always go outside of both the setup and loop functions.

03

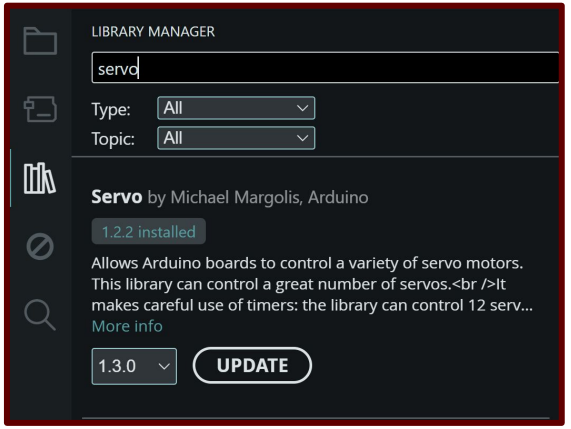
Libraries in Arduino

What is a Library?

A library is a prepackaged file that contains function definitions for the sketch author to use. The .h and .cpp files used in the previous examples are a pseudo-library. They contain the same file types and structure as a traditional library, but instead of being installed directly to the Arduino IDE, they must be downloaded and put in a specific location.



To add a library into the Arduino IDE, click this icon on the left bar and search the name of the library. Click 'install' once the library comes up. Try this with the 'Servo' library shown in the screenshot below



Including a Library

To save resources, C++ does not automatically initialize all libraries for every sketch. Instead, to use a library in a sketch, the library must be included at the top of the sketch using the following syntax:

```
#include <Servo.h>
```

Note that the name of the .h file is placed inside <>, while local .h files, are placed inside of “ ”. Quotations should only be used when the file being included is already in the same folder as the sketch.

Servo Library

The servo library is a very commonly used library when connecting servos directly to an Arduino. To find documentation on how to use a library, try one of the following methods:

1. Opening and analyzing example sketches.
2. Searching for documentation online.
   (E.g, a github page or readme file)

Opening an example sketch:

